

Ex.No: 13

PROGRAM

```
# (Save the notebook as backpropagation_ann.ipynb)
import pandas as pd
import numpy as np
from IPython.display import display
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
# Load Iris dataset (built into scikit-learn, no upload needed)
data = pd.read_csv("iris.csv")
print("First five records:")
display(data.head())
print("Dataset Shape:", data.shape, "-> 4 input features + 1 target column")
print("\nDataset Information:")
data.info()
print("\nClass distribution:")
print(data["Species"].value_counts())
```

Output:

First five records:

	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species
0	1	5.1	3.5	1.4	0.2	Iris-setosa
1	2	4.9	3.0	1.4	0.2	Iris-setosa
2	3	4.7	3.2	1.3	0.2	Iris-setosa
3	4	4.6	3.1	1.5	0.2	Iris-setosa
4	5	5.0	3.6	1.4	0.2	Iris-setosa

Dataset Shape: (150, 6) -> 4 input features + 1 target column

Dataset Information:

<class 'pandas.DataFrame'>

RangeIndex: 150 entries, 0 to 149

Data columns (total 6 columns):

#	Column	Non-Null Count	Dtype
0	Id	150 non-null	int64
1	SepalLengthCm	150 non-null	float64
2	SepalWidthCm	150 non-null	float64
3	PetalLengthCm	150 non-null	float64
4	PetalWidthCm	150 non-null	float64
5	Species	150 non-null	str

dtypes: float64(4), int64(1), str(1)

memory usage: 7.2 KB

```

Class distribution:
Species      *
Iris-setosa      50
Iris-versicolor  50
Iris-virginica   50
Name: count, dtype: int64

```

```

# Separate the 4 input features from the target column
feature_cols = ["SepalLengthCm", "SepalWidthCm", "PetalLengthCm", "PetalWidthCm"]
X = data[feature_cols]
y = data["Species"]
encoder = LabelEncoder()
y = encoder.fit_transform(y)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
# MLPClassifier trains the neural network using error backpropagation.
model = MLPClassifier(
    hidden_layer_sizes=(10,),
    activation="relu",
    solver="sgd",
    learning_rate_init=0.01,
    max_iter=1000,
    random_state=42
)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
print("Actual Values   :", y_test)
print("Predicted Values:", y_pred)
print("\nAccuracy:", accuracy_score(y_test, y_pred))
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))
print("\nClassification Report:")
print(classification_report(
    y_test, y_pred, target_names=encoder.classes_, zero_division=0
))

```

Output:

```
Results visualization:




|             | Actual \ Predicted | 0 | 1  | 2  | Support |
|-------------|--------------------|---|----|----|---------|
| Predicted 0 | 10                 | 0 | 0  | 0  | 10      |
| Predicted 1 | 0                  | 0 | 10 | 0  | 10      |
| Predicted 2 | 0                  | 0 | 0  | 10 | 10      |
| Accuracy    | 0.9666666666666667 |   |    |    |         |


```

```
Actual Values : [0 2 1 1 0 1 0 0 2 1 2 2 2 1 0 0 0 1 1 2 0 2 1 2 2 1 1 0 2 0]
Predicted Values: [0 2 1 1 0 1 0 0 2 1 2 2 2 1 0 0 0 1 1 2 0 2 1 2 2 2 1 0 2 0]
```

Accuracy: 0.9666666666666667

Confusion Matrix:

```
[[10  0  0]
 [ 0  9  1]
 [ 0  0 10]]
```

Classification Report:

	precision	recall	f1-score	support
Iris-setosa	1.00	1.00	1.00	10
Iris-versicolor	1.00	0.90	0.95	10
Iris-virginica	0.91	1.00	0.95	10
accuracy			0.97	30
macro avg	0.97	0.97	0.97	30
weighted avg	0.97	0.97	0.97	30

Predict a new sample (must have the same 4 feature columns, in the same order)

```
new_sample = pd.DataFrame(
    [[5.1, 3.5, 1.4, 0.2]], columns=feature_cols)
new_sample_scaled = scaler.transform(new_sample)
prediction = model.predict(new_sample_scaled)
print("New Sample Prediction:", encoder.inverse_transform(prediction)[0])
```

Output:

New Sample Prediction: Iris-setosa